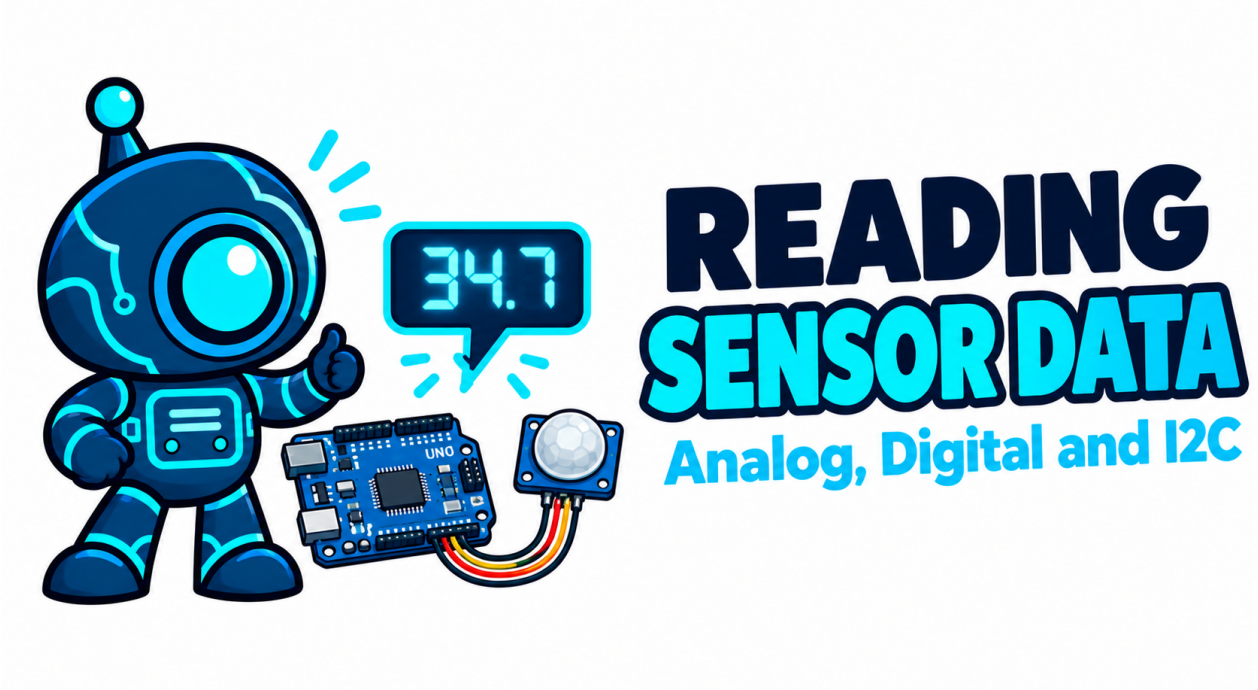
AI MECH LAB

Mechatronics Course — Beginner Stage



Module 3 · Topic 3.3

Reading Sensor Data with a Microcontroller

Reading Sensor Data with a Microcontroller

Topics 3.1 and 3.2 told you what sensors output and how to choose one. This topic is where you read them. The examples use an Arduino Uno-class board, which has 5 V logic and a 10-bit ADC. Other boards follow the same ideas with different numbers, such as a different ADC resolution, reference voltage or logic level, so check your own board's documentation before you copy any figures.

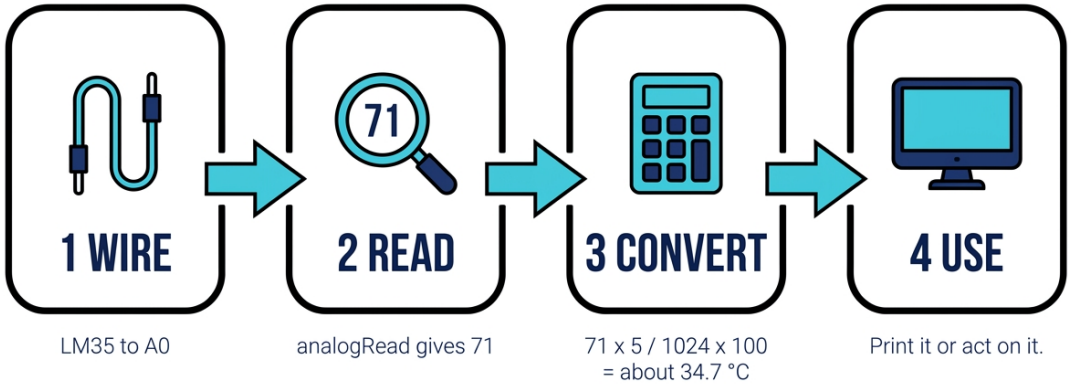


A first sensor reading: wire it, read it, convert it, then check it.

The four-step pattern

Every sensor reading, whatever the sensor, follows the same four steps.

1. **Wire** the sensor correctly: power, ground and signal.
2. **Read** the raw value, using the function that matches the signal type.
3. **Convert** the raw value into real units.
4. **Use** the result, either by printing it for debugging or by acting on it.

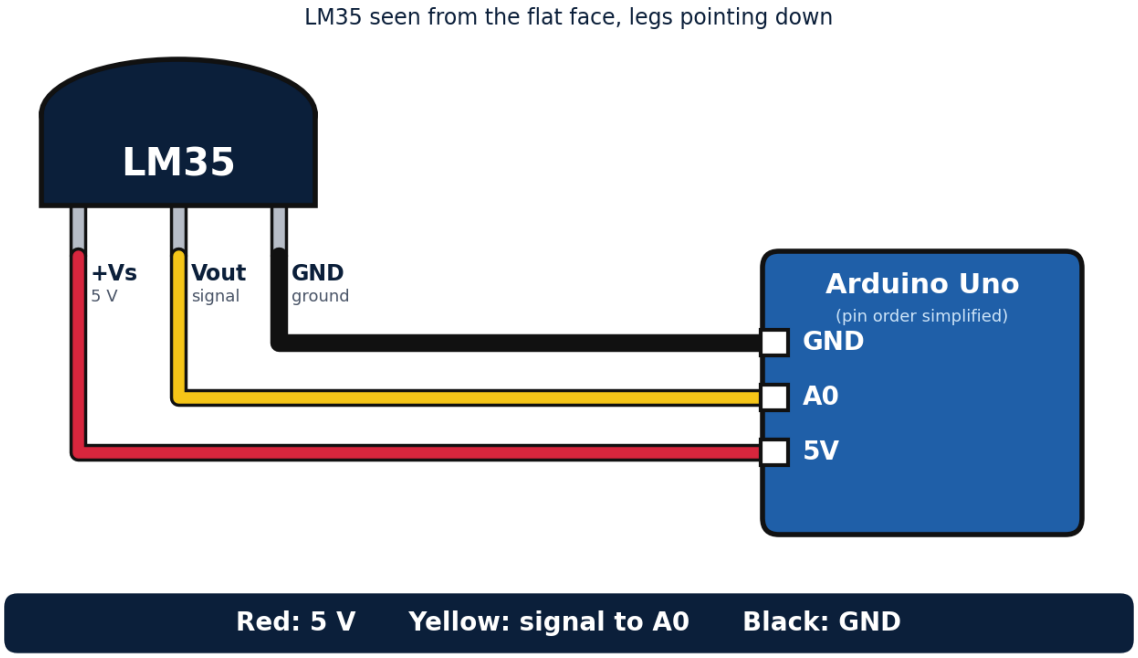


The four steps, shown with the LM35 example from this topic.

The Serial Monitor in the Arduino software is your window into step 4. Your sketch sends text to your computer with `Serial.print`, and you watch it appear. Open the monitor at 9600 baud, matching the `Serial.begin(9600)` line in the code, or you will see garbage. Every sketch also has two parts: `setup()` runs once at power-up, and `loop()` repeats forever.

Analog signals: reading an LM35

Connect the LM35's supply pin to 5 V, its ground pin to GND, and its output pin to analog input A0. Check the pin order in the datasheet, which is the skill from Topic 2.4, because wiring it backwards will heat the chip.



LM35 wiring: +Vs to 5 V, Vout to A0, GND to GND. Always confirm the pin order in your datasheet.

```
const int sensorPin = A0;

void setup() {
  Serial.begin(9600);
}

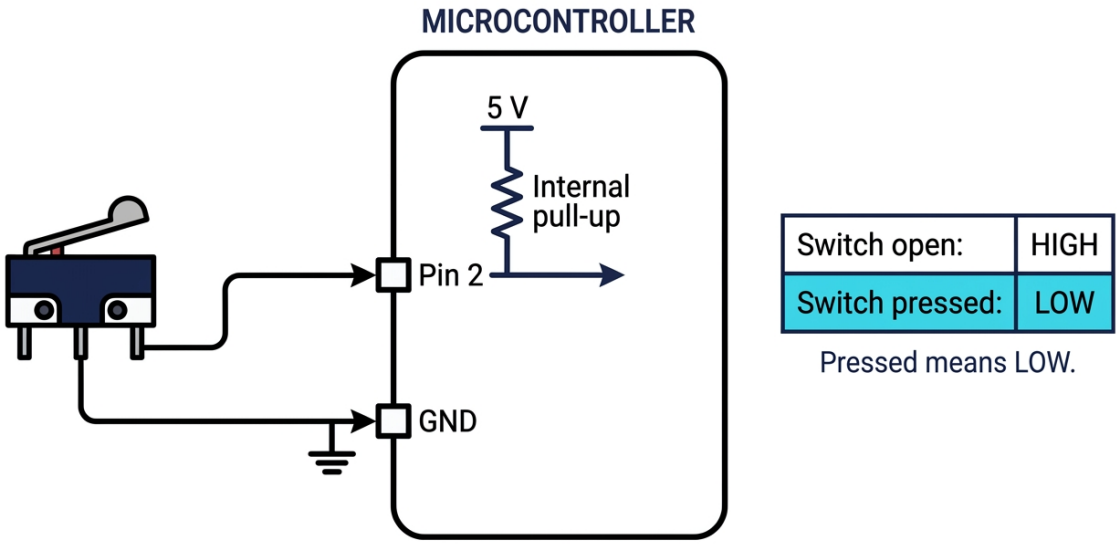
void loop() {
  int raw = analogRead(sensorPin);        // 0 to 1023
  float volts = raw * (5.0 / 1024.0);     // convert to volts
  float tempC = volts * 100.0;            // LM35: 10 mV per degree
  Serial.print(raw);
  Serial.print("  ");
  Serial.print(volts, 3);
  Serial.print(" V  ");
  Serial.print(tempC, 1);
  Serial.println(" C");
  delay(500);
}
```

`analogRead` returns a whole number from 0 to 1,023, which is the ADC from Topic 3.1. This is the same worked example you saw there. At 35.0 °C the sensor outputs 0.350 V, `analogRead` returns 71, the conversion gives 0.347 V, and the sketch prints 34.7 °C. Each ADC step here equals about 0.49 °C, so the readings move in half-degree jumps. That is the resolution limit, not a fault.

If you need finer steps, the Uno has an internal reference of nominally 1.1 V. Add `analogReference(INTERNAL);` to `setup()` and change the conversion to `raw * (1.1 / 1024.0)`. Each step then equals about 0.1 °C, though the top of the range drops to roughly 110 °C. The 1.1 V figure is nominal and varies from chip to chip, so check the result against a thermometer you trust.

Digital on/off signals: a limit switch

Connect one side of the switch to pin 2 and the other side to GND. Then let the board supply the pull-up resistor from Topic 3.2 for you with `INPUT_PULLUP`. With it enabled, the pin reads HIGH while the switch is open, and pressing the switch connects the pin to ground, so it reads LOW. The logic is inverted: **pressed means LOW.**



With the internal pull-up enabled, the pin reads HIGH when open and LOW when pressed.

Switches also bounce, so the sketch waits a few milliseconds and checks again before believing a change.

```
const int switchPin = 2;
bool lastState = HIGH;

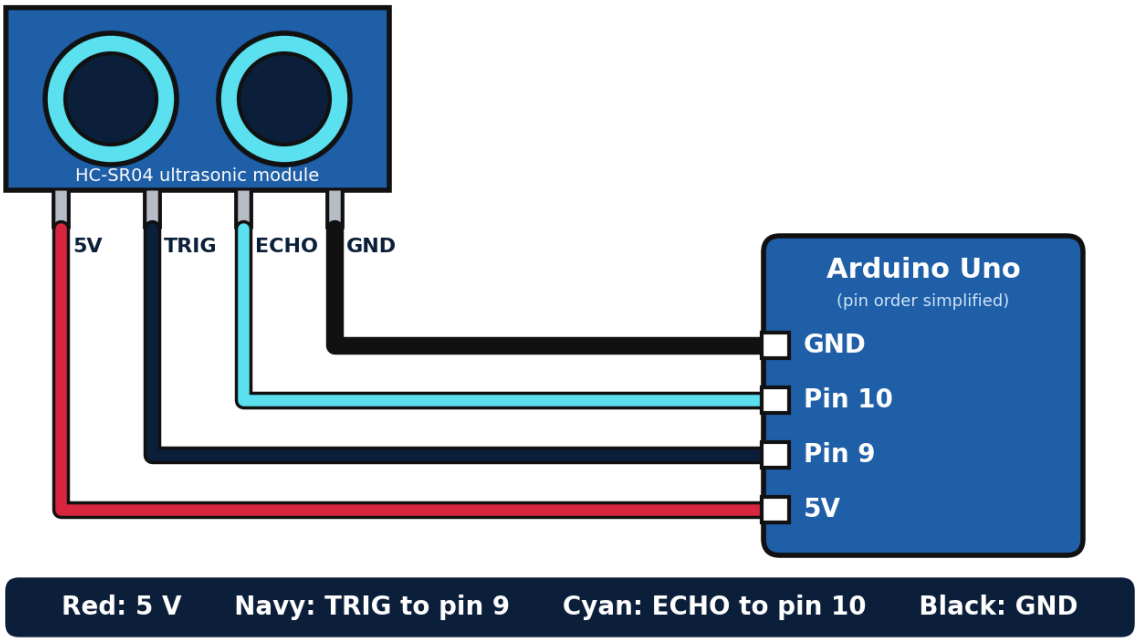
void setup() {
  Serial.begin(9600);
  pinMode(switchPin, INPUT_PULLUP);
}

void loop() {
  bool state = digitalRead(switchPin);
  if (state != lastState) {
    delay(20);                        // wait out the bounce
    state = digitalRead(switchPin);   // read again
    if (state != lastState) {
      lastState = state;
      if (state == LOW) {
        Serial.println("Pressed");
      } else {
        Serial.println("Released");
      }
    }
  }
}
```

The same `INPUT_PULLUP` setting usually supplies the pull-up that an A3144 Hall sensor needs. A PIR sensor's output is already a clean high or low, so a plain `digitalRead` on a pin set with `INPUT` is enough.

Pulse timing: an ultrasonic sensor

Connect the sensor's power to 5 V, ground to GND, TRIG to pin 9 and ECHO to pin 10. The Uno works at 5 V, so the echo pin needs no divider here. The divider from Topic 3.2 is for 3.3 V controllers.



Ultrasonic wiring: 5 V to 5V, TRIG to pin 9, ECHO to pin 10, GND to GND.

The sketch sends a 10-microsecond pulse on TRIG, then times how long ECHO stays high with `pulseIn`.

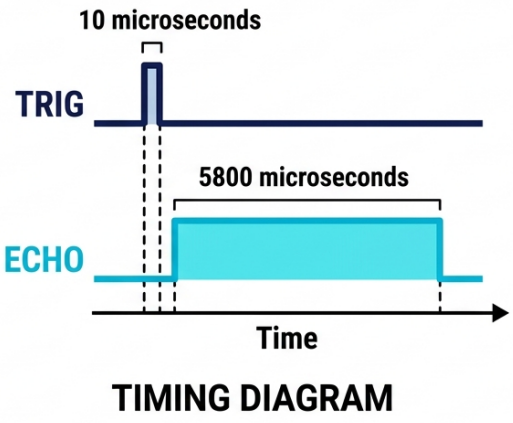
```
const int trigPin = 9;
const int echoPin = 10;

void setup() {
  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
}

void loop() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);       // 10 microsecond trigger pulse
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long duration = pulseIn(echoPin, HIGH, 30000);   // microseconds, 0 means no echo
  if (duration == 0) {
    Serial.println("No echo");
  } else {
    float cm = duration * 0.0343 / 2.0;
    Serial.print(cm, 1);
    Serial.println(" cm");
  }
  delay(100);
}
```

Sound travels about 343 m/s, which is 0.0343 cm per microsecond. The pulse covers the trip out and back, so you divide by 2. A 5,800-microsecond echo gives 5,800 x 0.0343 / 2, which is about 99.5 cm. The 30,000-microsecond timeout stops the sketch from waiting forever when nothing is in range.

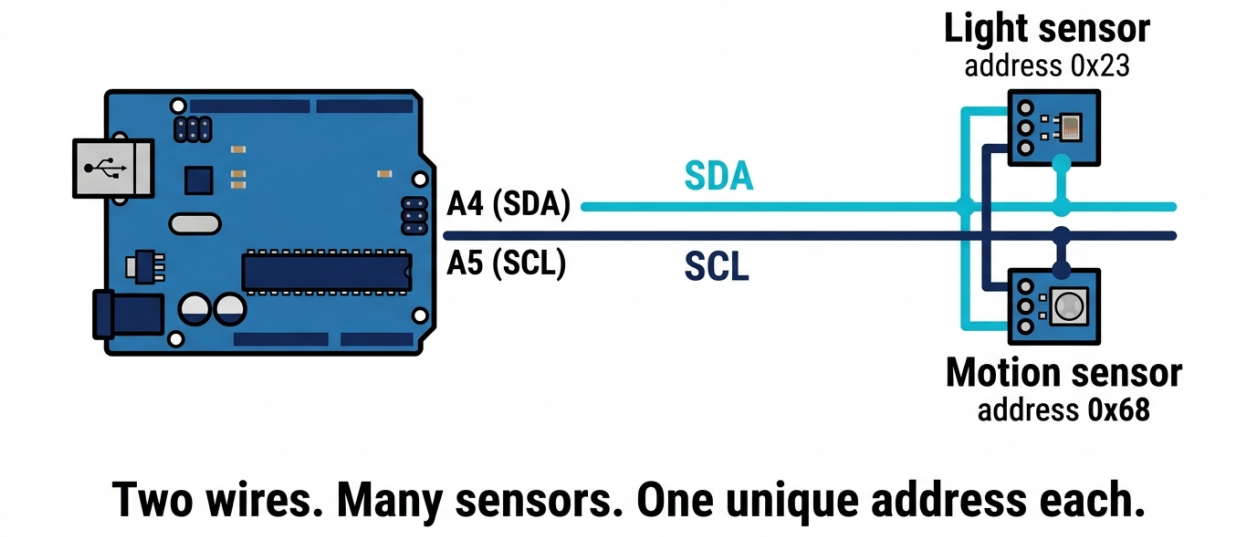


distance = 5800 x 0.0343 / 2 = about 99.5 cm

Trigger pulse, echo pulse and the distance calculation.

Digital data: I²C sensors

I²C sensors share two wires, SDA for data and SCL for the clock, and every device on the bus has its own address. On the Uno, SDA is pin A4 and SCL is pin A5. Power and ground go to the sensor as usual. Before you read anything, scan the bus to confirm the sensor is actually there. This catches wiring mistakes in seconds.



Two wires, many sensors, one unique address each. Power and ground are not shown.

```
#include <Wire.h>

void setup() {
  Serial.begin(9600);
  Wire.begin();
  for (byte address = 1; address < 127; address++) {
    Wire.beginTransmission(address);
    if (Wire.endTransmission() == 0) {
      Serial.print("Device found at 0x");
      Serial.println(address, HEX);
    }
  }
}

void loop() {
}
```

A BH1750 shows up at 0x23, or 0x5C if its address pin is pulled high. The BH1750 chip itself is a 3.3 V part, as you saw in Topic 3.2, so make sure your breakout board handles that before you connect it to 5 V. With the address confirmed, reading it takes a short sequence: send a command that starts measuring, wait, then ask for two bytes and join them into one number.

```
#include <Wire.h>

const byte bh1750 = 0x23;

void setup() {
  Serial.begin(9600);
  Wire.begin();
  Wire.beginTransmission(bh1750);
  Wire.write(0x10);                    // continuous high-resolution mode
  Wire.endTransmission();
  delay(180);                          // first measurement takes time
}

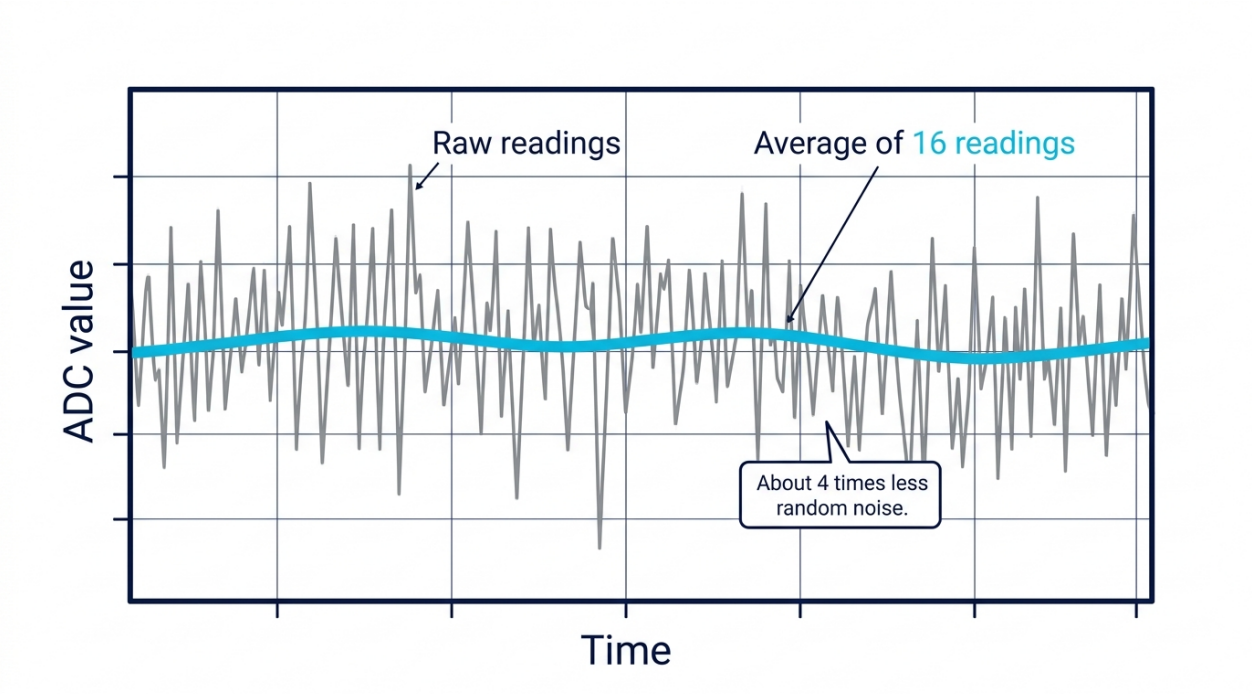
void loop() {
  Wire.requestFrom((int)bh1750, 2);
  if (Wire.available() == 2) {
    byte high = Wire.read();
    byte low = Wire.read();
    unsigned int raw = (high << 8) | low;
    float lux = raw / 1.2;             // datasheet conversion factor
    Serial.print(lux, 0);
    Serial.println(" lux");
  }
  delay(500);
}
```

Confirm the command byte and the divide-by-1.2 factor against the BH1750 datasheet for your own part, because a wrong command gives silence or nonsense. In practice, most people read I^2C and other protocol sensors through a library, which hides these byte-level details. The DS18B20 from Topic 3.2, for example, is normally read through the OneWire and DallasTemperature libraries, which you install from the Library Manager. A library does not remove the wiring rules, so the DS18B20 still needs its 4.7 kΩ pull-up.

Taming noise

Even with a constant input, analog readings wobble by a count or two, because of the noise you met in Topic 3.1. The simplest fix is to take several readings and average them.

```
float readAverage(int pin, int samples) {
  long total = 0;
  for (int i = 0; i < samples; i++) {
    total += analogRead(pin);
    delay(2);
  }
  return total / (float)samples;
}
```



Averaging 16 readings smooths the random wobble, at the cost of a slower response.

Averaging 16 readings cuts random noise by roughly a factor of four, since it falls with the square root of the number of samples. It does nothing about a sensor that is consistently wrong, and it slows the response, which is the response-time trade-off from Topic 3.1. For a slow temperature signal that is a good bargain. For a fast vibration signal it is not.

When the readings look wrong

Work through this list before blaming the sensor.

1. **No common ground.** The sensor and the board must share ground, or the readings drift randomly.
2. **Wrong pin or mode.** `analogRead` only works on the analog pins, A0 to A5 on the Uno. Digital inputs need `pinMode` set correctly.
3. **Wrong supply voltage.** A 3.3 V sensor on 5 V can fail quietly or permanently.
4. **Missing pull-up.** An input that floats gives random values.
5. **Integer maths.** In Arduino code, `5 / 1024` is 0, because dividing two whole numbers gives a whole number. Write `5.0 / 1024.0`.
6. **Converting before checking.** Print the raw value first. If the raw number is wrong, no conversion will fix it.
7. **Not measuring.** Put your multimeter on the sensor pin and compare it with what the code says. If the multimeter reads 0.350 V and your code says 0.347 V, the code is right. This is the habit from Topic 2.4 paying off.

Quick reference

Signal type	Typical sensor	Arduino approach
Analog voltage	LM35, Sharp infrared, potentiometer	`analogRead`, then convert
Digital on/off	Limit switch, PIR, A3144	`digitalRead` with `INPUT` or `INPUT_PULLUP`
Pulse width	Ultrasonic	`pulseIn`, then convert time to distance
I^2C data	BH1750, MPU-6050	`Wire` library, or a sensor library
1-Wire data	DS18B20	OneWire and DallasTemperature libraries
Pulse counting	Encoder	Interrupts or a library, beyond this beginner topic

> ***Going deeper:*** *the Robotics course revisits this from the point of view of a robot that reads many sensors at once and combines them into a picture of its surroundings.*

Where this leaves you

You can now wire a sensor, read it with the function that matches its signal, convert the raw number into real units and check it against your multimeter. That completes Module 3. You know what a sensor is, which one to pick, and how to read it. Next comes Module 4, where we move to the other end of the loop, starting with Topic 4.1, **What Is an Actuator?**, and then use these sensors to drive motors in the capstone.